COMP3153/9153
Algorithmic Verification

Lecture 9: Verification Games

Games in Computer Science

Definition

A game is an interactive process between a number of players.

Games in Computer Science

Definition

A game is an interactive process between a number of players.

Games in Computer Science

- Visualize complex concepts
  - Logical formula evaluation
  - Non-determinism and alternation
  - Program semantics

Games in Computer Science

Definition

A game is an interactive process between a number of players.

Games in Computer Science

- Visualize complex concepts
  - Logical formula evaluation
  - Non-determinism and alternation
  - Program semantics
- Define complex concepts

Games in Computer Science

Definition

A game is an interactive process between a number of players.

Games in Computer Science

- Visualize complex concepts
  - Logical formula evaluation
  - Non-determinism and alternation
  - Program semantics
- Define complex concepts
- Model complex interactions
  - Auction theory
  - Resource management

Games for formal verification

Verification games

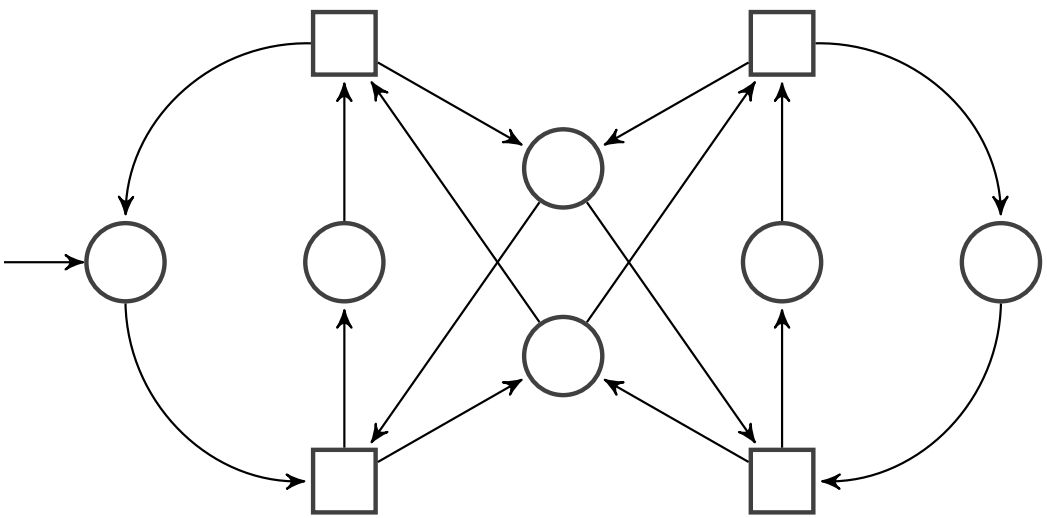
The types of games we are interested in:

- Two players (Adam and Eve)
- Turn based
- Perfect information
- Zero sum

Verification games

Definition

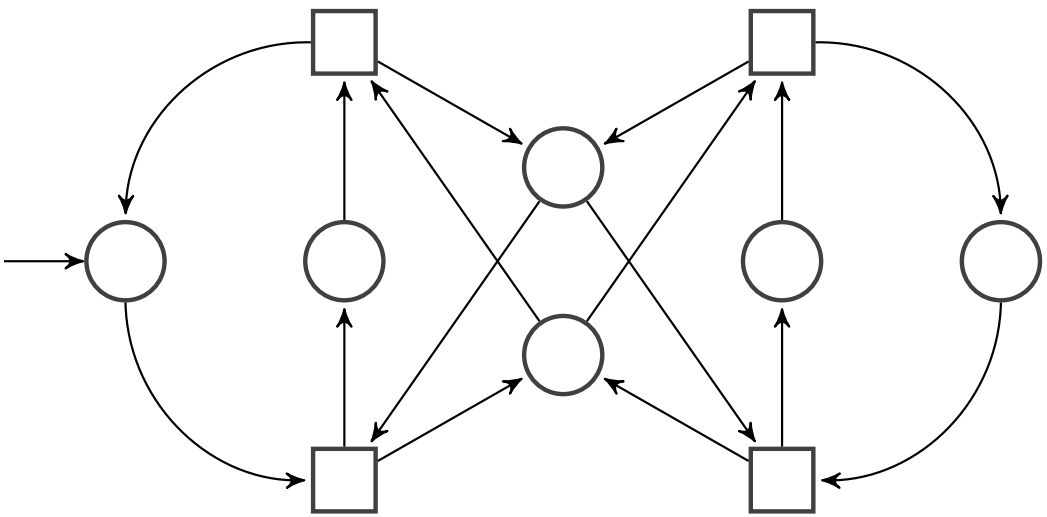
An arena is an automaton whose states are partitioned into two sets: V_E and V_A.



Verification games

Definition

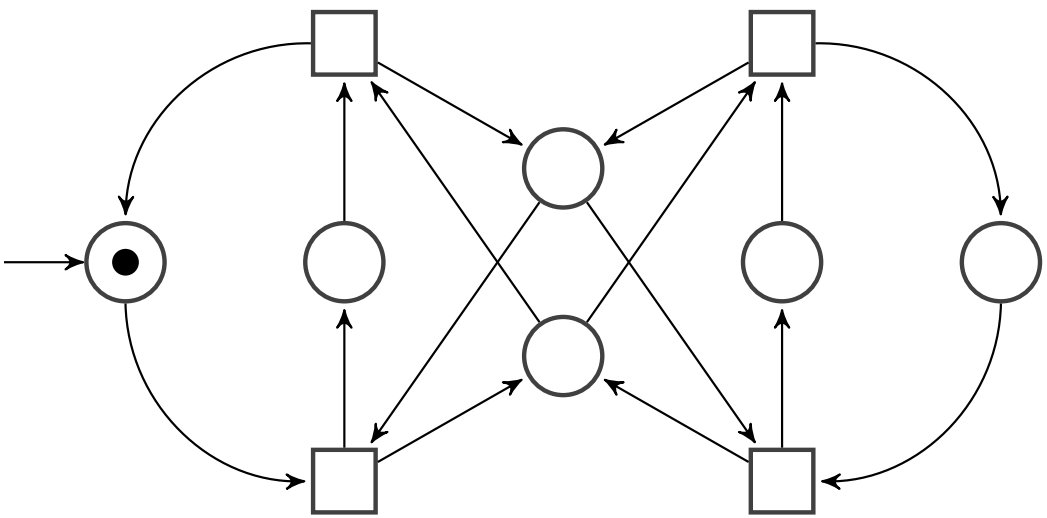
A game consists of an arena $\mathcal{A}$ together with a set Win of [possibly infinite] runs (sequences of states)



Verification games

Definition

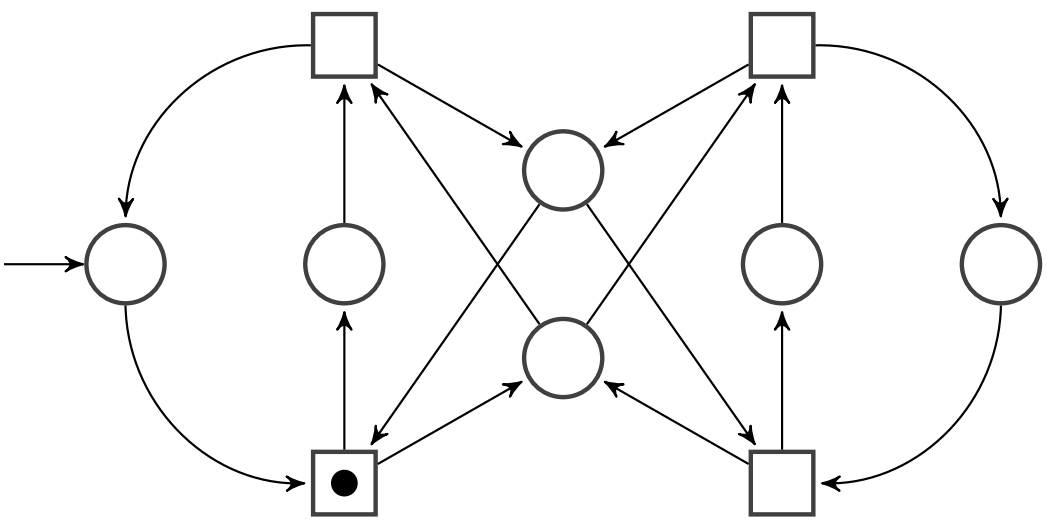
A game consists of an arena $\mathcal{A}$ together with a set Win of [possibly infinite] runs (sequences of states)
Two players move a token around $\mathcal{A}$. Eve choosing if the token is in V_E, Adam choosing otherwise.



Verification games

Definition

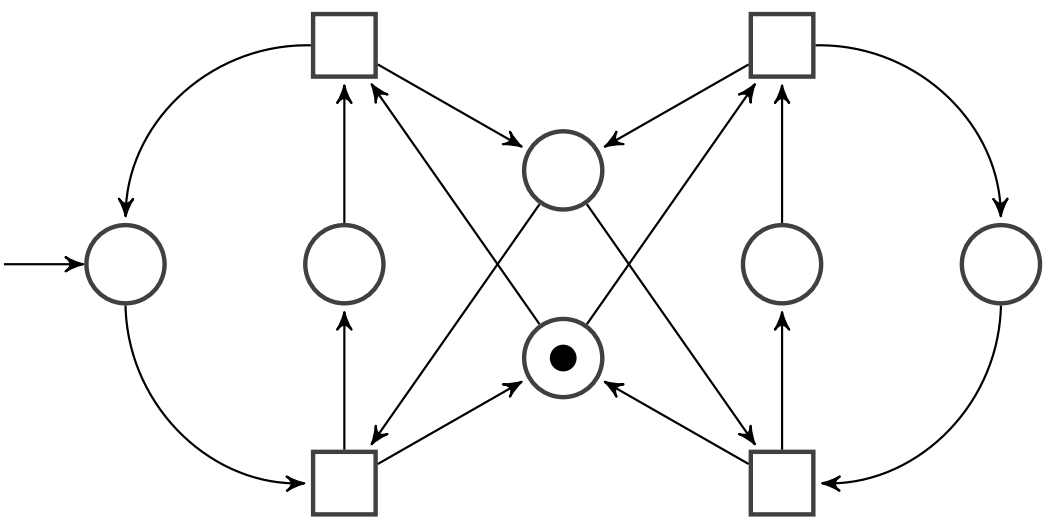
A game consists of an arena $\mathcal{A}$ together with a set Win of [possibly infinite] runs (sequences of states)
Two players move a token around $\mathcal{A}$. Eve choosing if the token is in V_E, Adam choosing otherwise.



Verification games

Definition

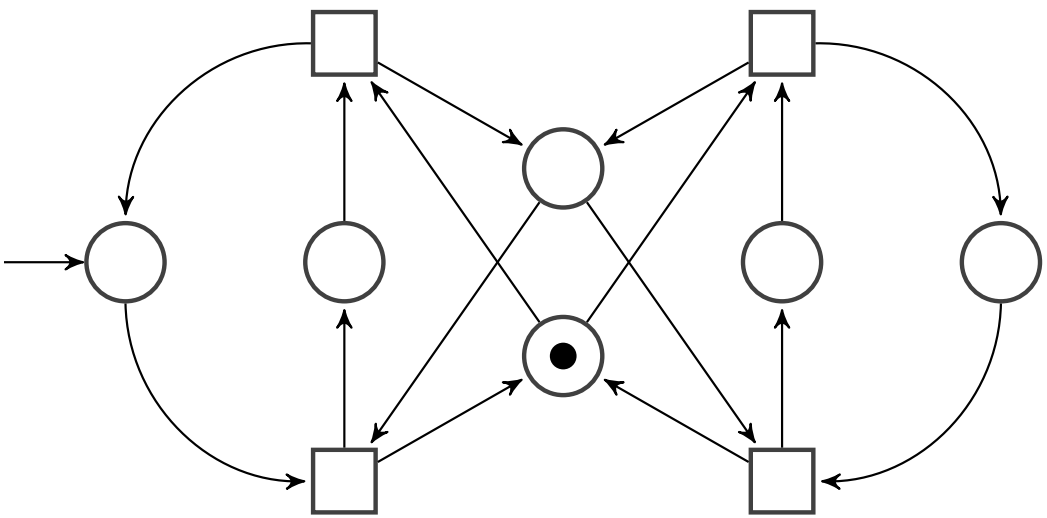
A game consists of an arena $\mathcal{A}$ together with a set Win of [possibly infinite] runs (sequences of states)
Two players move a token around $\mathcal{A}$. Eve choosing if the token is in V_E, Adam choosing otherwise.



Verification games

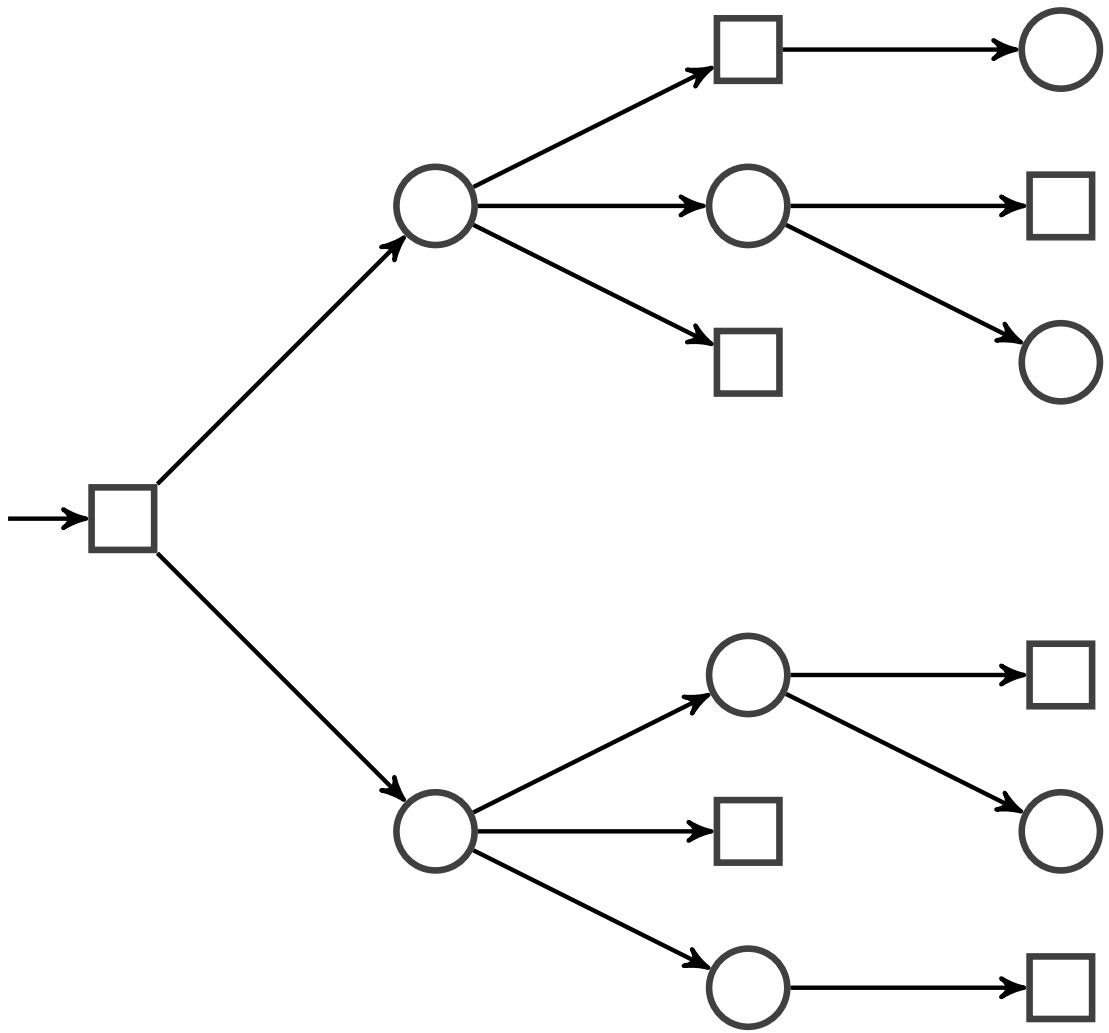
Definition

A game consists of an arena $\mathcal{A}$ together with a set Win of [possibly infinite] runs (sequences of states)
Two players move a token around $\mathcal{A}$. Eve choosing if the token is in V_E, Adam choosing otherwise.
This defines a run ρ. Eve wins if $\rho \in \text{Win}$



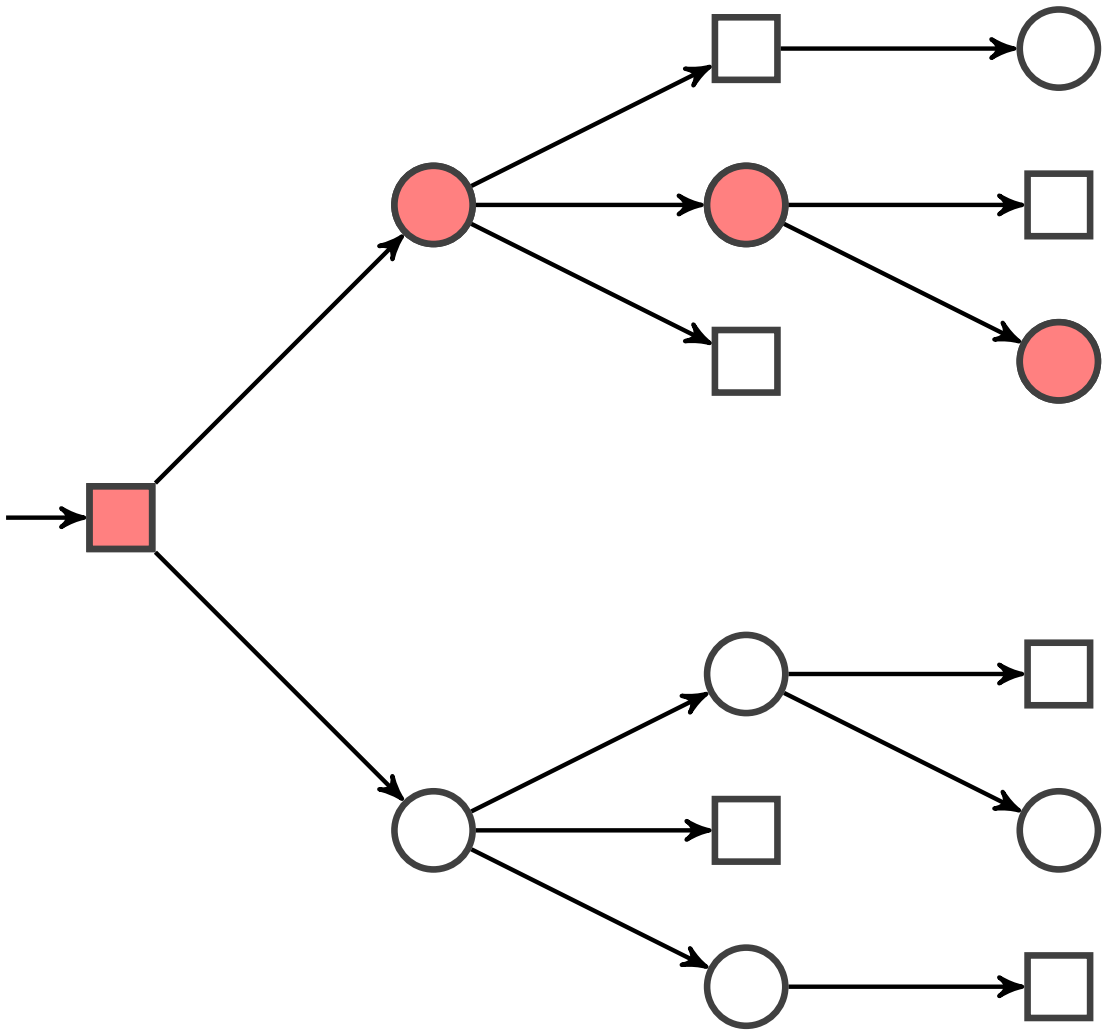
Example

Eve wins if and only if Adam cannot move.



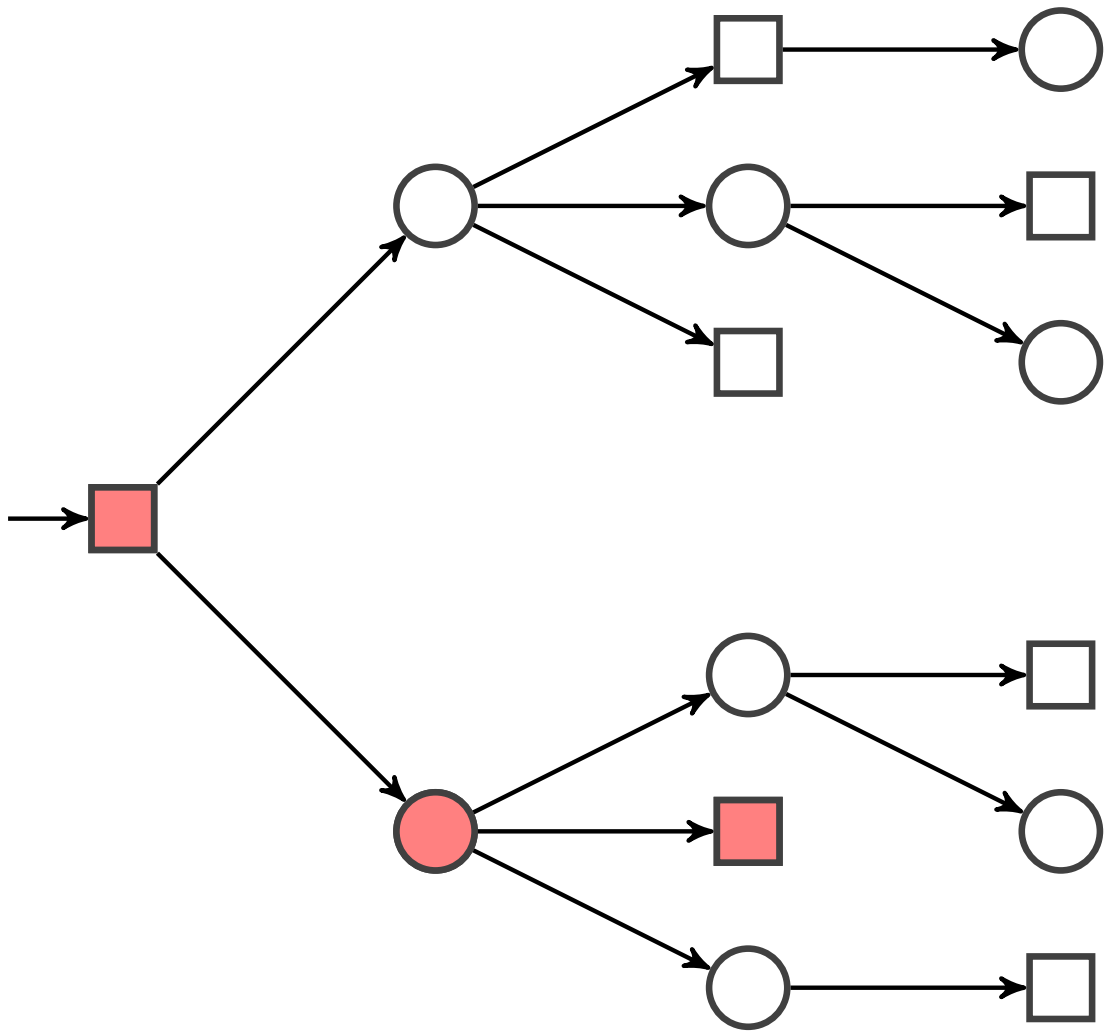
Example

Eve wins if and only if Adam cannot move.



Example

Eve wins if and only if Adam cannot move.



Strategies

Definition

A strategy for Eve (Adam) is a function that maps partial runs ending in a state $v \in V_E$ (V_A) to a state v' such that $(v, v') \in \delta$.

Strategies

Definition

A strategy for Eve (Adam) is a function that maps partial runs ending in a state $v \in V_E$ (V_A) to a state v' such that $(v, v') \in \delta$.
A run ρ is consistent with a strategy σ if $\rho_{i+1} = \sigma(\rho|_i)$ for all suitable i.

Strategies

Definition

A strategy for Eve (Adam) is a function that maps partial runs ending in a state $v \in V_E$ (V_A) to a state v' such that $(v, v') \in \delta$.
A run ρ is consistent with a strategy σ if $\rho_{i+1} = \sigma(\rho|_i)$ for all suitable i.
A strategy σ is winning for Eve (Adam) if every run consistent with the strategy is in Win.
A strategy σ is winning for Adam if every run consistent with the strategy is not in Win.

NB

Given a pair of strategies σ, τ for Adam and Eve, there is exactly one run that is consistent with σ and τ.

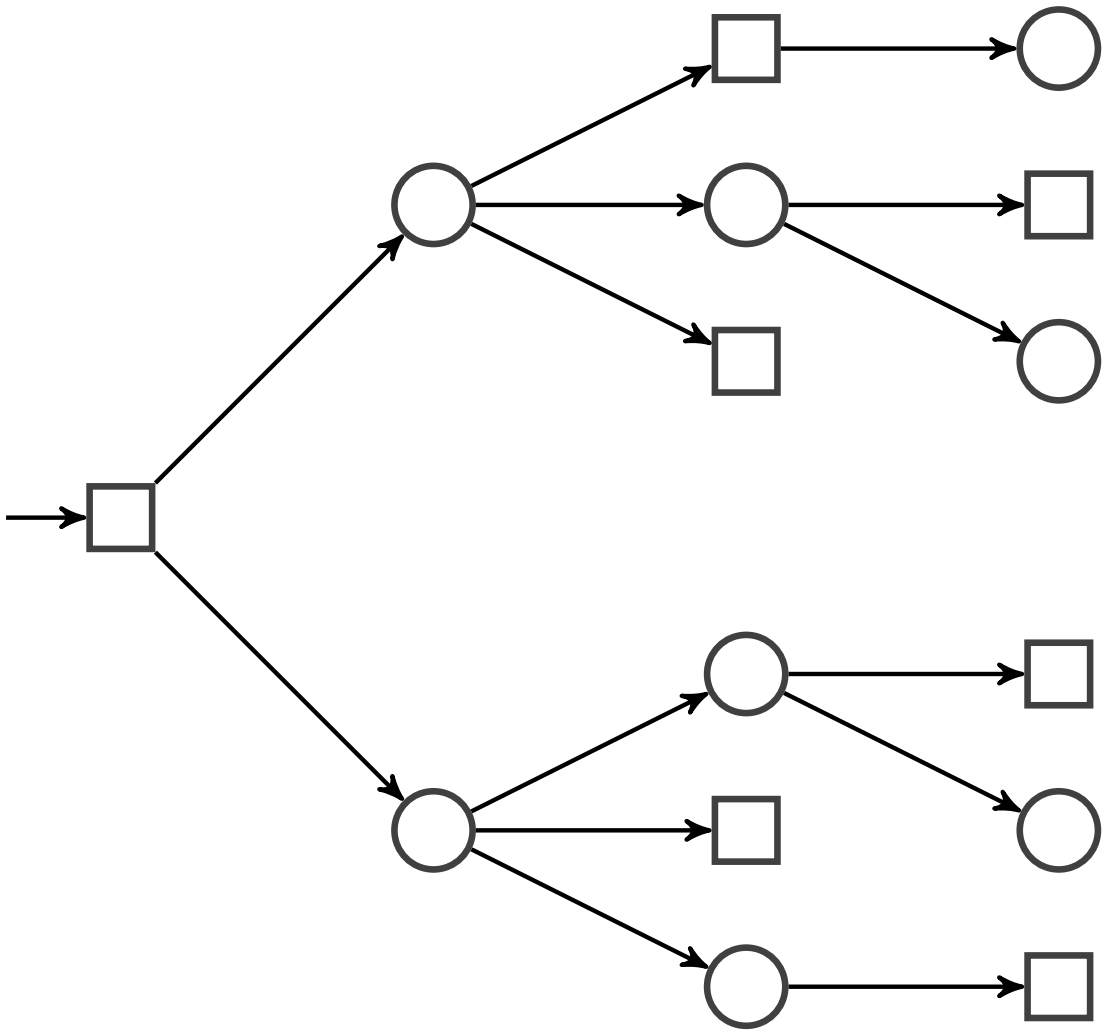
Memory

Definition

A finite memory strategy is a strategy that can be implemented with a finite automaton.
A strategy that only depends on the current state is memoryless or positional.

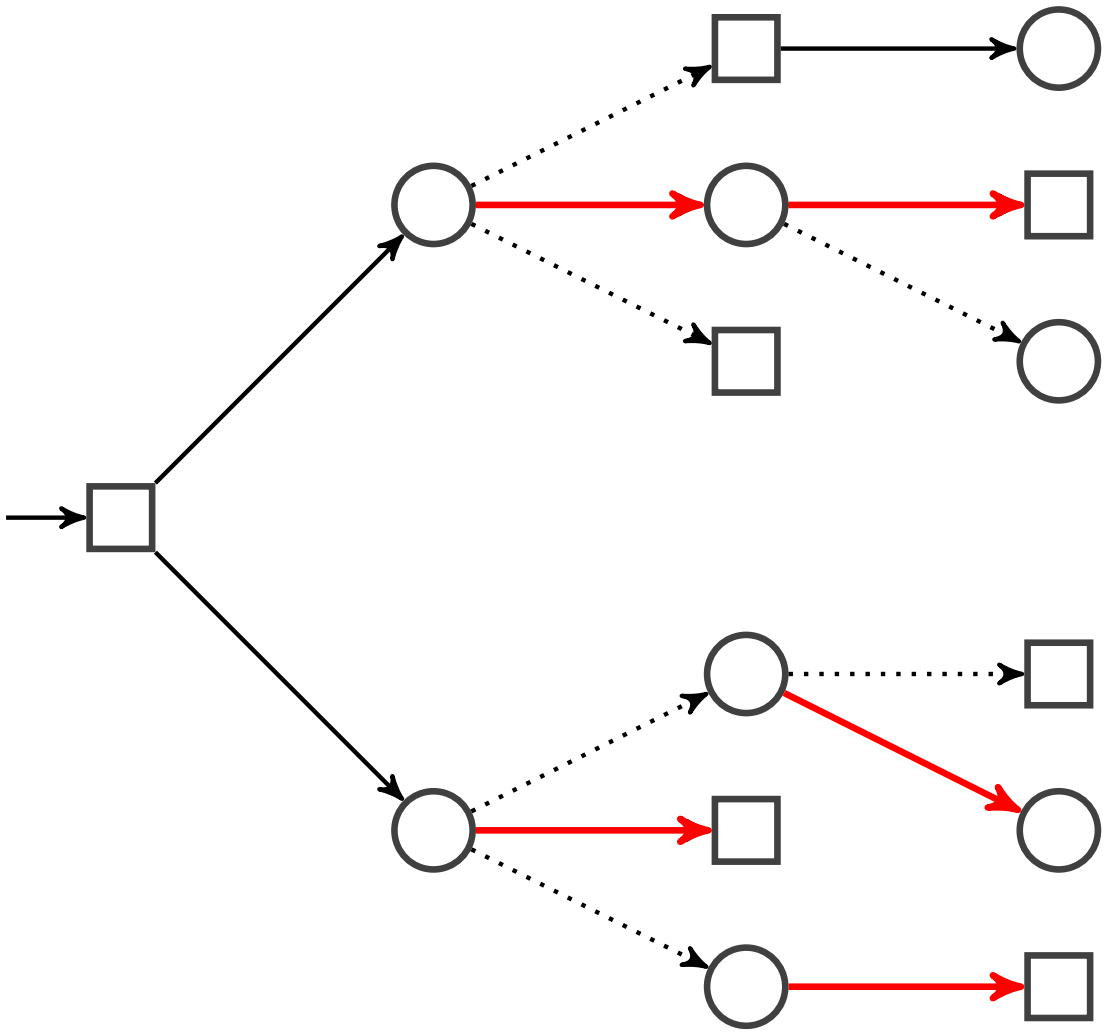
Example

Eve wins if and only if Adam cannot move.



Example

Eve wins if and only if Adam cannot move.



Determinacy

Theorem

If Win is a set of finite runs, then one player has a winning strategy.

Determinacy

Theorem

If Win is a set of finite runs, then one player has a winning strategy.
If Win is a Borel set, then one player has a winning strategy.

Determinacy

Theorem

If Win is a set of finite runs, then one player has a winning strategy.
If Win is a Borel set, then one player has a winning strategy.

Algorithmic problem

Given a game $(\mathcal{A}, \mathrm{Win})$, which player has a winning strategy?

Determinacy

Theorem

If Win is a set of finite runs, then one player has a winning strategy.
If Win is a Borel set, then one player has a winning strategy.

Algorithmic problem

Given a game $(\mathcal{A}, \mathrm{Win})$, which player has a winning strategy?

Generally undecidable! (Win could have a complex definition)

Infinite duration games

We are primarily concerned with games played for an infinite number of turns.

Definition

A winning condition Win is ω-regular if whether or not $\rho \in \text{Win}$ is determined by the set of states visited infinitely often.

$$\liminf(\rho) = \liminf(\rho') \implies \rho \in \text{Win} \Leftrightarrow \rho' \in \text{Win}$$

Infinite duration games

We are primarily concerned with games played for an infinite number of turns.

Definition

A winning condition Win is ω-regular if whether or not $\rho \in \text{Win}$ is determined by the set of states visited infinitely often.

$$\liminf(\rho) = \liminf(\rho') \implies \rho \in \text{Win} \Leftrightarrow \rho' \in \text{Win}$$

Theorem

If Win is ω-regular then one player has a finite-memory winning strategy.

ω-regular winning conditions

Common ways to define *Win*

Name	Given by	Condition
Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F \neq \emptyset$
co-Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F = \emptyset$

ω-regular winning conditions

Common ways to define *Win*

Name	Given by	Condition
Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F \neq \emptyset$
co-Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F = \emptyset$
Rabin	$\{(E_i, F_i) \ : \ E_i, F_i \subseteq Q\}$	$\exists i,\ E_i \cap \liminf(\rho) \neq \emptyset$ and $F_i \cap \liminf(\rho) = \emptyset$

ω-regular winning conditions

Common ways to define *Win*

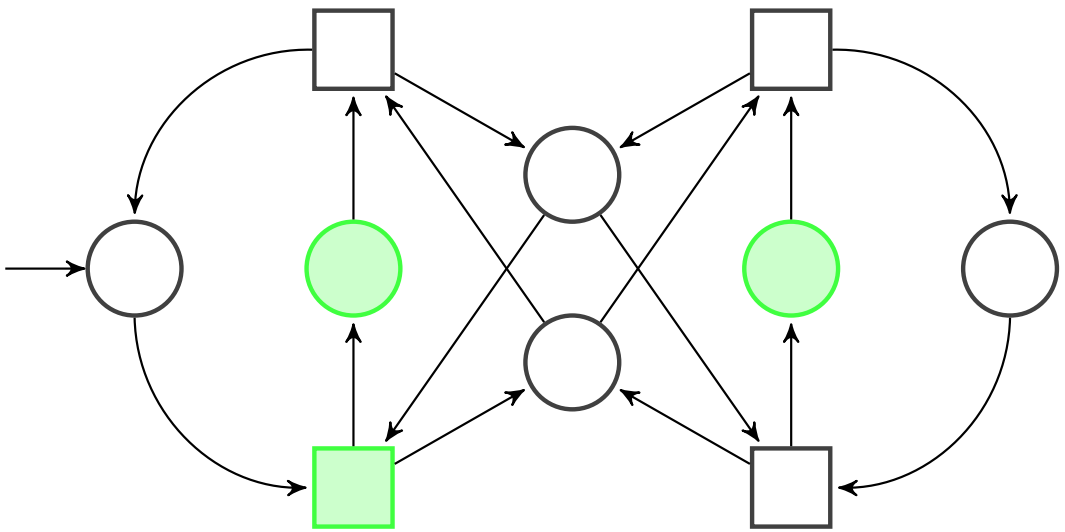
Name	Given by	Condition
Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F \neq \emptyset$
co-Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F = \emptyset$
Rabin	$\{(E_i, F_i) : E_i, F_i \subseteq Q\}$	$\exists i,\ E_i \cap \liminf(\rho) \neq \emptyset$ and $F_i \cap \liminf(\rho) = \emptyset$
Muller	$\mathcal{F} \subseteq 2^Q$	$\liminf(\rho) \in \mathcal{F}$

ω-regular winning conditions

Common ways to define *Win*

Name	Given by	Condition
Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F \neq \emptyset$
co-Büchi	$F \subseteq Q$	$\liminf(\rho) \cap F = \emptyset$
Rabin	$\{(E_i, F_i) \,:\, E_i, F_i \subseteq Q\}$	$\exists i,\ E_i \cap \liminf(\rho) \neq \emptyset$ and $F_i \cap \liminf(\rho) = \emptyset$
Muller	$\mathcal{F} \subseteq 2^Q$	$\liminf(\rho) \in \mathcal{F}$
Parity	$p : Q \to \mathbb{N}$	$\liminf(p(\rho))$ is even

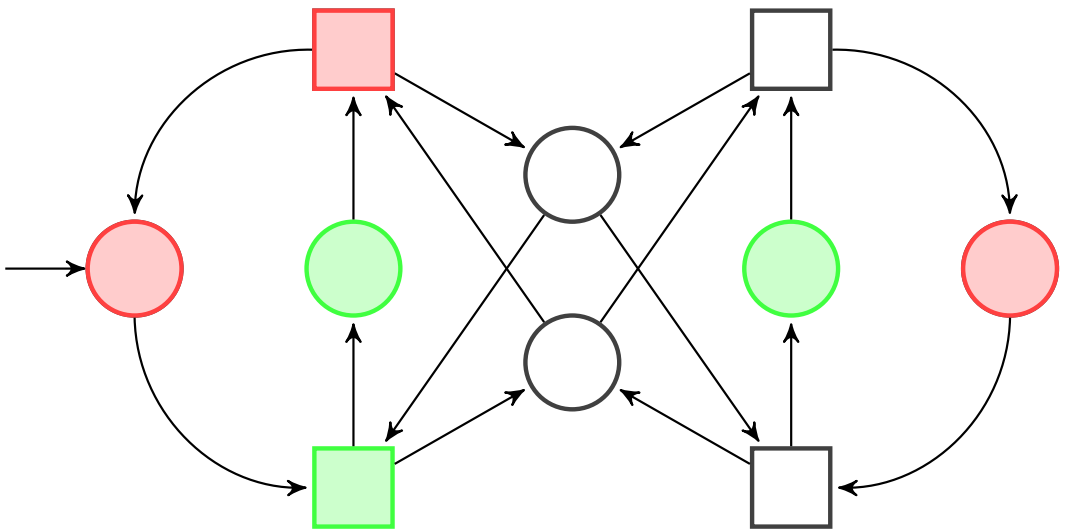
Examples



Büchi game

Eve wins if Green is visited infinitely often

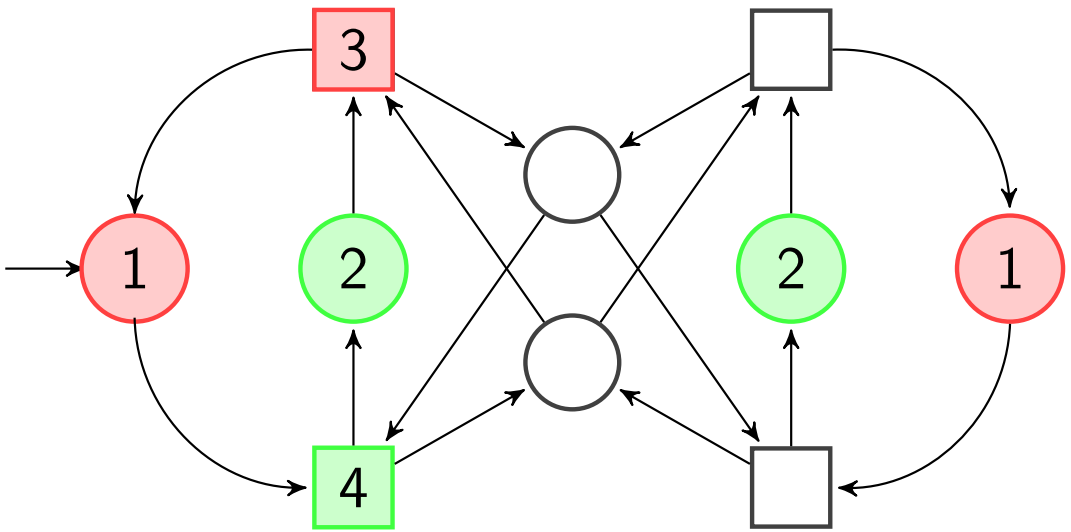
Examples



Rabin game

Eve wins if Green is visited infinitely often and Red is visited finitely often

Examples



Parity game

Eve wins if the minimum value seen infinitely often is even

Relationships between winning condition types

In terms of describing families of sets:

$$\begin{matrix}\text{Büchi} \\ \text{co-Büchi}\end{matrix} \Rightarrow \text{Parity} \Rightarrow \text{Rabin} \Rightarrow \text{Muller}$$

Relationships between winning condition types

In terms of describing families of sets:

$$\begin{matrix}\text{Büchi}\\ \text{co-Büchi}\end{matrix} \Rightarrow \text{Parity} \Rightarrow \text{Rabin} \Rightarrow \text{Muller}$$

Theorem

For every Muller game $\mathcal{G} = (\mathcal{A}, \mathcal{F})$ there exists a parity game $\mathcal{G}' = (\mathcal{A}', p)$ such that Eve wins $\mathcal{G}$ if and only if she wins $\mathcal{G}'$

Positional strategies

Theorem

- *Let $\mathcal{G}$ be a Rabin game. If Eve has a winning strategy, then she has a positional winning strategy.*
- *Let $\mathcal{G}$ be a parity game. Whichever player has a winning strategy has a positional winning strategy.*

Positional strategies

Theorem

- *Let $\mathcal{G}$ be a Rabin game. If Eve has a winning strategy, then she has a positional winning strategy.*
- *Let $\mathcal{G}$ be a parity game. Whichever player has a winning strategy has a positional winning strategy.*

Corollary

- *Deciding if Eve wins a Rabin game is in NP.*
- *Deciding if Eve wins a parity game is in $\mathrm{NP} \cap \mathrm{coNP}$.*

Positional strategies

Theorem

- *Let $\mathcal{G}$ be a Rabin game. If Eve has a winning strategy, then she has a positional winning strategy.*
- *Let $\mathcal{G}$ be a parity game. Whichever player has a winning strategy has a positional winning strategy.*

Corollary

- *Deciding if Eve wins a Rabin game is in NP.*
- *Deciding if Eve wins a parity game is in $\mathrm{NP} \cap \mathrm{coNP}$.*

Open question

Can the winner of a parity game be determined in polynomial time?

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\varphi \quad = p \wedge (q \vee (p \vee \neg r))$$

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\varphi \quad = p \wedge (q \vee (p \vee \neg r))$$

Bottom-up approach:

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\begin{aligned} \varphi &= p \wedge (q \vee (p \vee \neg r)) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \neg \texttt{true})) \end{aligned}$$

Bottom-up approach:

- Substitue values in

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\begin{aligned}\varphi &= p \wedge (q \vee (p \vee \neg r)) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \neg \texttt{true})) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \texttt{false}))\end{aligned}$$

Bottom-up approach:

- Substitue values in
- Evaluate operators when all operands are known

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\begin{aligned}\varphi &= p \wedge (q \vee (p \vee \neg r)) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \neg \texttt{true})) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \texttt{false})) \\ &= \texttt{true} \wedge (\texttt{false} \vee \texttt{true})\end{aligned}$$

Bottom-up approach:

- Substitue values in
- Evaluate operators when all operands are known

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\begin{aligned}\varphi &= p \wedge (q \vee (p \vee \neg r)) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \neg \texttt{true})) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \texttt{false})) \\ &= \texttt{true} \wedge (\texttt{false} \vee \texttt{true}) \\ &= \texttt{true} \wedge \texttt{true}\end{aligned}$$

Bottom-up approach:

- Substitue values in
- Evaluate operators when all operands are known

Evaluating logical formulas

Given $p = \texttt{true}$, $q = \texttt{false}$, $r = \texttt{true}$, evaluate:

$$\begin{aligned}\varphi &= p \wedge (q \vee (p \vee \neg r)) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \neg \texttt{true})) \\ &= \texttt{true} \wedge (\texttt{false} \vee (\texttt{true} \vee \texttt{false})) \\ &= \texttt{true} \wedge (\texttt{false} \vee \texttt{true}) \\ &= \texttt{true} \wedge \texttt{true} \\ &= \texttt{true}\end{aligned}$$

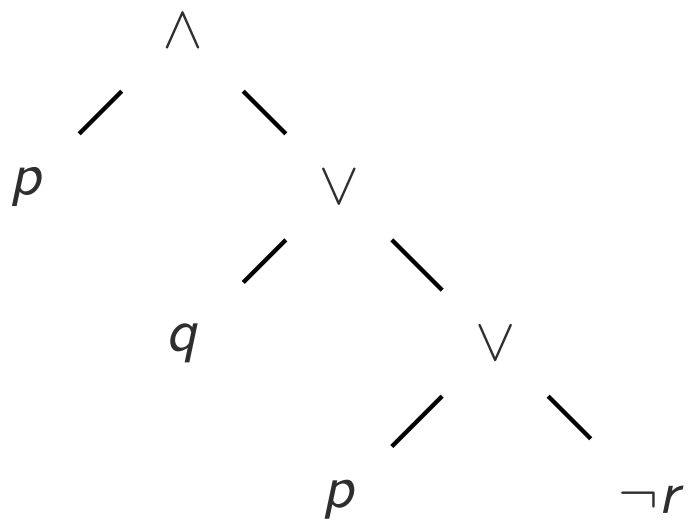
Bottom-up approach:

- Substitue values in
- Evaluate operators when all operands are known

Using games to evaluate formulas

Top-down approach: Use games!

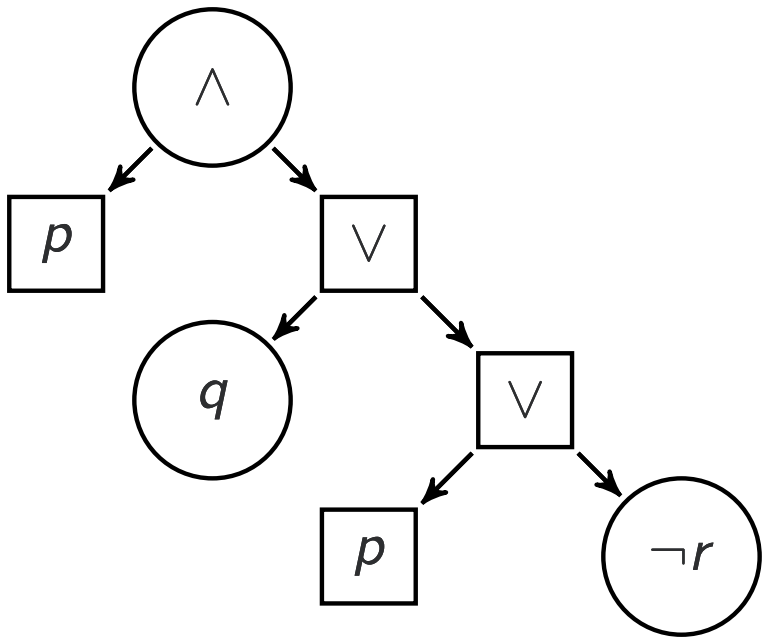
$p = \texttt{true}$
$q = \texttt{false}$
$r = \texttt{true}$



Using games to evaluate formulas

Top-down approach: Use games!

$p = \texttt{true}$
$q = \texttt{false}$
$r = \texttt{true}$



Eve wins if and only if Adam cannot move

Formula game

Definition

Given a propositional formula φ (in NNF) and a valuation $v : \mathcal{P} \to \{\texttt{true}, \texttt{false}\}$, the arena of the formula game is defined recursively as follows:

- If $\varphi = p$ or $\varphi = \neg p$ and $v(\varphi) = \texttt{true}$ then it is a single Adam node
- If $\varphi = p$ or $\varphi = \neg p$ and $v(\varphi) = \texttt{false}$ then it is a single Eve node
- If $\varphi = \psi_1 \wedge \psi_2$ then it consists of an Adam node that chooses between the game for ψ_1 and the game for ψ_2
- If $\varphi = \psi_1 \vee \psi_2$ then it consists of an Eve node that chooses between the game for ψ_1 and the game for ψ_2

The winning condition for the formula game is that Eve wins if and only if Adam cannot move (i.e. runs that end in a terminal Adam state).

Using games to evaluate formulas

Can we deal with $\neg$ at a higher level?

Using games to evaluate formulas

Can we deal with $\neg$ at a higher level?
Reverse the roles of the players!

Using games to evaluate formulas

Can we deal with $\neg$ at a higher level?
Reverse the roles of the players!

Definition

If $\mathcal{A}$ is an arena, the dual arena, $\overline{\mathcal{A}}$, is the arena obtained by exchanging V_E and V_A.

Using games to evaluate formulas

Can we deal with $\neg$ at a higher level?
Reverse the roles of the players!

Definition

If $\mathcal{A}$ is an arena, the dual arena, $\overline{\mathcal{A}}$, is the arena obtained by exchanging V_E and V_A.

Observation

Eve does not have a winning strategy in $(\mathcal{A}, \mathrm{Win})$ if and only if she has a winning strategy in $(\overline{\mathcal{A}}, \overline{\mathrm{Win}})$

Using games to evaluate formulas

Principle extends to other logics:

- First order logic:
  - $\exists x$ – Eve chooses x;
  - $\forall x$ – Adam chooses x

Using games to evaluate formulas

Principle extends to other logics:

- First order logic:
  - $\exists x$ – Eve chooses x;
  - $\forall x$ – Adam chooses x
- Modal logic:
  - **A X** – Adam chooses the successor;
  - **E X** – Eve chooses the successor

Using games to evaluate formulas

Principle extends to other logics:

- First order logic:
  - $\exists x$ – Eve chooses x;
  - $\forall x$ – Adam chooses x
- Modal logic:
  - **A X** – Adam chooses the successor;
  - **E X** – Eve chooses the successor

Question

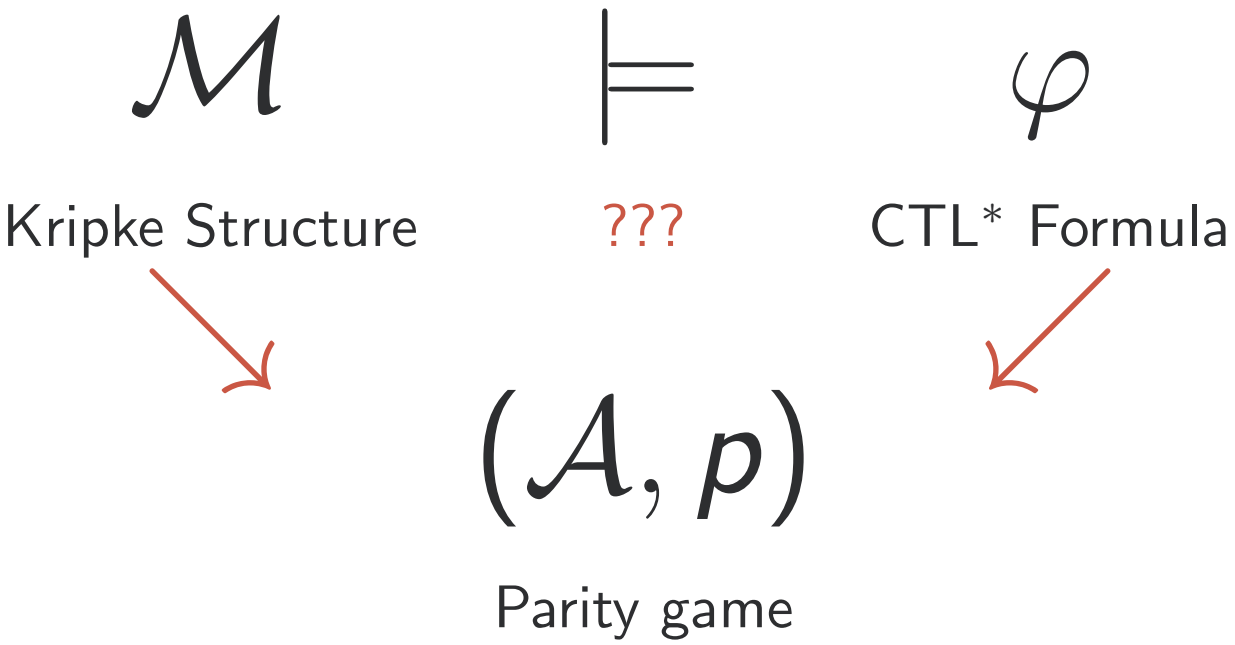
What about temporal logics?

Using games to model check

$\mathcal{M}$ $\models$ φ

Kripke Structure ??? CTL* Formula

Using games to model check



Using games to model check

General idea:

- States consist of subformulas of φ and states of $\mathcal{M}$
- Build the game recursively as with the formula game for propositional formulas
- For temporal operators, the appropriate player chooses the next state
- Eventuality is enforced by the parity condition (levels correspond to nested untils)

Bibliography

- Grädel, Thomas, Wilke (ed.), Automata, Logics and Infinite Games, 2002
- Paul Hunter, Complexity and Infinite Games (PhD thesis), 2008